

The Skateboard: a Minimum Framework for AI-Assisted Porting of Fortran 2008 to GPU, with Professional Assurances

Pared down from `answers.html`; three-month scope

Original scope: July 23, 2026; implementation note: September 6, 2026

Document status. This is the original three-month scope proposal, retained for its research rationale and planning history. Sections labeled “original” or “planned” describe that proposal rather than guarantees of the current repository. The implementation note below records the principal differences as of September 2026. For operational and evidence claims, the source tree, the companion architecture, and the version-2 evidence contract are authoritative over an older generated PDF.

1 Purpose and the original demonstration claim

The full framework in `answers.html` describes eleven components and fourteen requirements. It is the right design for a program that, like DARPA TRACTOR, must port many codes repeatedly. Our near-term goal is different: in three months, demonstrate on one 30,000-line Fortran 2008 code that an AI agent can carry out GPU ports *with little human intervention*, and that the result is *correct*, *effective*, and *repeatable* to a standard an engineering organization would accept. This document specifies the least framework that supports that claim — the skateboard, in the sense of the smallest artifact that rolls end to end, rather than a wheel from the eventual car.

The paring rule is simple, and it comes straight from the TRACTOR evidence. Every component in the full design does one of two things: it *produces evidence* for a claim (correct, effective, repeatable), or it *improves throughput and scale* (planners, ensembles, provenance graphs, compiler matrices, prompt optimization). We keep everything in the first category, essentially intact, because the assurances are the demonstration. We cut or manually substitute everything in the second category, because with one code, a handful of candidate regions, and three months, a human can do those jobs in hours. The demonstration loses nothing it needs: TRACTOR’s central finding is that outcomes are driven by orchestration and oracles, not by generation — and oracles are precisely what we are keeping.

This framing also serves the client story about how professional AI use differs from personal AI use. The agent is the same in both settings. What differs is the harness: a developer on a personal project accepts “the tests I remembered to run passed on my machine,” while an engineering organization requires oracles the agent cannot edit, gates that make forbidden outcomes impossible, and a record that lets a reviewer reconstruct why each change was accepted. The skateboard *is* that harness, at minimum size; the demo makes the difference visible by construction.

2 What the original proposal retained

Four things are kept whole. Each maps to one of the three claims.

1. The correctness oracle: capture-replay tests plus a tolerance comparator (claim: correct). We instrument each candidate region during runs on representative datasets, capture every input the region reads and every output it writes (arguments, module variables, any residual COMMON state), and generate a standalone replay driver. A comparator checks replayed outputs against captured outputs under per-quantity tolerances — absolute, relative, or units-in-the-last-place — because a GPU port legitimately changes floating-point results (fused multiply-add contraction, reduction reordering, different device math libraries). Tolerances are calibrated honestly and cheaply: we measure the spread the CPU code already exhibits across compilers and optimization/FMA settings, and set acceptance bands from that observed sensitivity. One representative dataset is *held out*: the agent never sees it, and it runs only at acceptance time. TRACTOR found failures concentrated at semantic comparison and translators overfitting to visible tests; the held-out set is the cheapest defense and we will not skip it. End-to-end golden-output runs (with the same tolerance discipline) back the region-level tests.

2. Hard gates (claims: correct and little-human-intervention). Three mechanical gates, none of which requires judgment: (a) *device-execution proof* — `OMP_TARGET_OFFLOAD=MANDATORY` so silent host fallback is a runtime error, plus a profiler-trace assertion that kernels launched, because a directive port can pass every functional test while never touching the GPU; (b) *read-only oracle* — captured data, tolerances, and the comparator live outside the agent’s writable tree, so “make the tests pass” cannot degenerate into “edit the tests”; any tolerance-change proposal is a distinct artifact routed to a human; (c) *sanitizer pass* — NVIDIA `compute-sanitizer` (memcheck, racecheck, initcheck) must run clean. Gates substitute for continuous human vigilance; they are what makes low intervention safe to claim.

3. The honest performance measurement (claim: effective). Acceptance requires beating the best CPU configuration — best flags, all cores of the socket via the code’s existing parallelism — on production-sized inputs, end to end, net of transfers, with repeated runs reported as distributions. We also run the naïve automatic port first (do concurrent with `-stdpar=gpu` and managed memory) so the agent’s work is measured against the floor a compiler flag already provides, not against zero. TRACTOR’s naïve baselines were embarrassingly competitive; ours may be too, and reporting that honestly strengthens rather than weakens the demonstration.

4. A bounded, scripted loop with a ledger (claims: repeatable and little-human-intervention). The orchestration is a deterministic script, not agent free-play: fixed stage order (build → device proof → sanitize → correctness → performance), a fixed iteration cap per stage, and a two-rung strategy ladder (standard parallelism with managed memory first; OpenMP `target` with explicit data regions second). Every attempt is a git branch with a one-line ledger entry: region, strategy, model, oracle verdicts, timings. Repeatability is then demonstrated, not asserted: we re-run the entire pipeline from a clean checkout three times per region and report success rate, intervention count, and variance in the achieved speedup. The ledger doubles as the intervention counter — the “little human intervention” claim becomes a measured number with an audit trail, which is exactly the kind of statement an engineering organization can put in a report.

3 Current implementation relative to the proposal

The repository now implements the evidence path, but not every automation and performance claim proposed above. An interactive Pi coding agent edits a configured working copy and calls a gateway reference monitor. The gateway, rather than a deterministic autonomous orchestrator, enforces separate onboarding and porting action tables. Retry counts and the strategy ladder are client policy;

there is no global iteration cap or held-out-query budget.

Submissions become immutable Git commits. A conservative SESE analyzer checks each candidate tree, and a later edit requires a new tree-scoped verdict. The analyzer does not establish complete call, alias, or effect closure; hidden Fortran state and the replay boundary remain scientific review obligations.

Ledger schema version 2 records strict SHA-256 subjects and materials. Framed file-set hashes bind paths and bytes without concatenation ambiguity. Current claims depend on the evidence-policy implementation, strategies, manifest, tolerances, properties, datasets, original-reference contract, and reviewed or observed executor and oracle identities. Claims after a build also carry the complete executable digest cohort. Changed inputs make older claims readable history rather than current evidence.

The trusted builder supervisor uses Docker to create a disposable job for each submitted build or execution. Jobs have no network or service token, use a read-only image, and receive only selected attempt data. Submitted processes run unprivileged; compiler tracing and GPU profiling write through a protected observer path. Later actions execute only binaries whose current digests match the protected build record. This establishes observed compiler execution and binary continuity, not complete final-binary provenance: a submitted Makefile could still overwrite an output after a legitimate compiler invocation.

Port acceptance currently requires candidate-scoped SESE, replay build and GPU execution, clean `memcheck`, `racecheck`, and `initcheck`, visible and held-out numerical regression, whole-program regression, valid repeated timing, and declared properties when present. Timing validates 5–100 returned durations and outputs and checks every port repetition against the baseline program outputs. It records measurements; it does not presently impose the proposal’s “must beat the best CPU configuration” speedup threshold.

Onboarding requires nine tree-scoped results: manifest validity; builds under both reviewed strategies; capture, replay, determinism, and timing; agreement with a preserved original program reference; mutation self-check; and declared properties. Property requests accept a Hypothesis ceiling of 100–10,000 examples and record separately observed replay executions. Mutation requests accept a limit of 1–10,000. Seeds are signed 64-bit integers.

The held-out oracle returns verdict-only receipts to the agent, but repeated queries can leak information. The Docker daemon, builder supervisor, job image, host kernel, and GPU driver remain in the trusted computing base. CPU boundary qualification does not qualify GPU profiling, sanitizers, or performance; those claims require the full qualification on the target NVIDIA host.

4 The original pared architecture and loop

Five components replace eleven. One is stochastic (the agent); the rest are scripts and fixtures a small team can build in weeks because every hard dependency is an existing tool: profiling via HPCToolkit or `perf` plus Nsight Systems; unit scaffolding via pFUnit; capture-replay adapted from the KGen/PSyclone-extraction pattern; the toolchain via `nvfortran` on one GPU node.

Two deliberate simplifications inside the loop deserve a note. *Candidate selection is human*: with one code, an engineer reading one profile picks the two to four demonstration regions in an afternoon, and we choose regions dominated by plain arrays and clean loop nests — deferring derived types with allocatable components, which are the hard case for any state-capture tool. This is scope control, not evasion, and we say so in the report. *Preconditioning is agent-proposed but conservatively validated*:

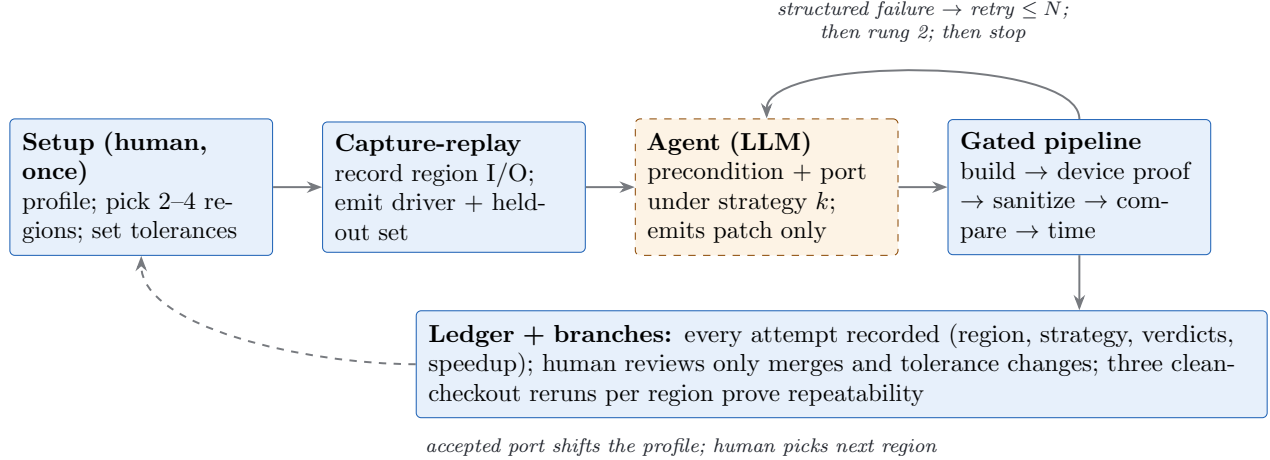


Figure 1: The loop proposed in July 2026. The current gateway enforces the gate dependencies, while retry and strategy budgets remain client policy.

the agent performs the de-COMMON-ing and **intent** annotation for its region as ordinary patches, the full existing test suite must pass after each preconditioning patch on the CPU path, and these patches get human review at merge like any other. The compiler-grade transform library from the full design is deferred, traded for region choice and suite coverage.

5 Planned human touchpoints

Exactly four, all logged: (1) initial region selection and tolerance sign-off; (2) review of any proposal to change a tolerance or accept a semantic difference the comparator surfaces — expected rarely, and valuable when it happens, since it usually means the port exposed a latent defect such as argument aliasing or an uninitialized variable; (3) merge review per accepted port; (4) a go/no-go per region when the agent exhausts its ladder and caps. Everything else — generation, repair against compiler errors, comparator discrepancies, and profiler feedback — runs unattended. The report can then say: M regions ported, N total human interventions of these four kinds, everything else autonomous, with the ledger as evidence.

6 Components deferred by the original proposal

Table 1 lists every full-design component that the skateboard drops or substitutes, the consequence of the cut, and the condition under which it should be restored — so the three-month artifact reads as the first increment of the full design rather than a different design.

7 Original three-month plan and proposed evidence pack

Month 1 — oracles before agents. Reproducible containerized build and CI on the existing test suite; profile on representative datasets; select regions and calibrate tolerances (human touchpoint 1); build the capture-replay generator and comparator for region 1; run the naïve `-stdpar` baseline and the honest multicore CPU baseline. Milestone: replay driver for region 1 passes against the CPU

Full-design component	Disposition	Cost, and when to restore
Planner (ranked regions, predicted ceilings)	Human, once	None for 2–4 regions. Restore when porting whole codes or many codes.
Deterministic preconditioner	Agent + suite + review	Slower, more review load. Restore (Flang/PSyclone-based) at scale, where LLM per-line error rates bite.
Multi-strategy ensemble	Two-rung ladder, sequential	May miss the best variant per region. Restore when GPU budget allows parallel strategy runs.
Provenance store (MVIR-like)	Git branches + CSV ledger	Adequate for one campaign. Restore for multi-code auditability.
Roofline service	Back-of-envelope, once	Weaker “gap to ceiling” attribution. Restore for tuning-heavy work.
Compiler/vendor matrix	One toolchain, one GPU	No portability claim. Restore before any multi-platform commitment.
Prompt optimization; model plurality	One model, versioned prompts	Possibly leaving quality on the table. Cheap to add later behind the same interface.
Evaluation battery (B01 analogue)	The code is the battery	No cross-code generality claim. Build the battery if this becomes a program.

Table 1: Everything cut is throughput or scale. Nothing cut weakens the correct/effective/repeatable evidence for the demonstrated regions — but the demonstration claims exactly those regions, on exactly one platform, and the report should say so plainly.

code under the tolerance policy, from a clean checkout, in CI. If month 1 slips, the schedule slips — nothing downstream is real without this.

Month 2 — one region, fully assured. Stand up the gated pipeline and the bounded agent loop; port region 1; iterate on structured feedback plumbing (this is where TRACTOR says the leverage is); first clean-checkout rerun. Milestone: region 1 accepted through all gates with a measured end-to-end speedup, and one rerun reproducing it.

Month 3 — breadth and the repeatability numbers. Regions 2–3 (and 4 if cheap); three clean-checkout reruns per region; assemble the evidence pack; write the report, including the personal-versus-professional framing of Section 1.

The evidence pack is the point of the exercise, so we name its contents now: the tolerance policy with its calibration data; per-region comparator reports on visible and held-out sets; device-execution traces; sanitizer logs; the baseline-versus-port timing distributions; the ledger with intervention counts; and the rerun variance table. Every sentence of the final report’s claims should trace to an artifact in this pack — that traceability, more than any single speedup number, is what distinguishes the engineering-organization mode of AI use from the personal one.

Principal risks. (1) Capture-replay engineering exceeds estimates for the chosen regions — mitigated by region selection (plain arrays first) and by the month-1 milestone acting as an early warning. (2) The naïve baseline wins on our regions — then the demonstration reframes, honestly and still favorably, as “AI verified and hardened the cheap port and proved where effort is unnecessary”; the assurance machinery is the deliverable either way. (3) GPU node availability and driver churn — pin everything in month 1 and keep exclusive access for measurement runs. (4) The existing test suite is weaker than assumed, undermining preconditioning validation — detected in month 1 by coverage measurement; the fallback is narrowing regions to code the replay tests themselves cover.